

Unified Modeling Tutorial

ADVANCE PROGRAMMING

In this tutorial, we will be looking to make the UML class diagram of a basic banking management system. The description of this system is as follows:

Each bank is identified by its code, and defines an address (assume single branch for each bank). Every bank will have a set of customers, accounts, and ATM booths. The bank has the functionality to print a list of all the accounts that it maintains. An account in this bank is identified by the account number, and the balance in the account. Money can be deposited into, or withdrawn from the accounts.

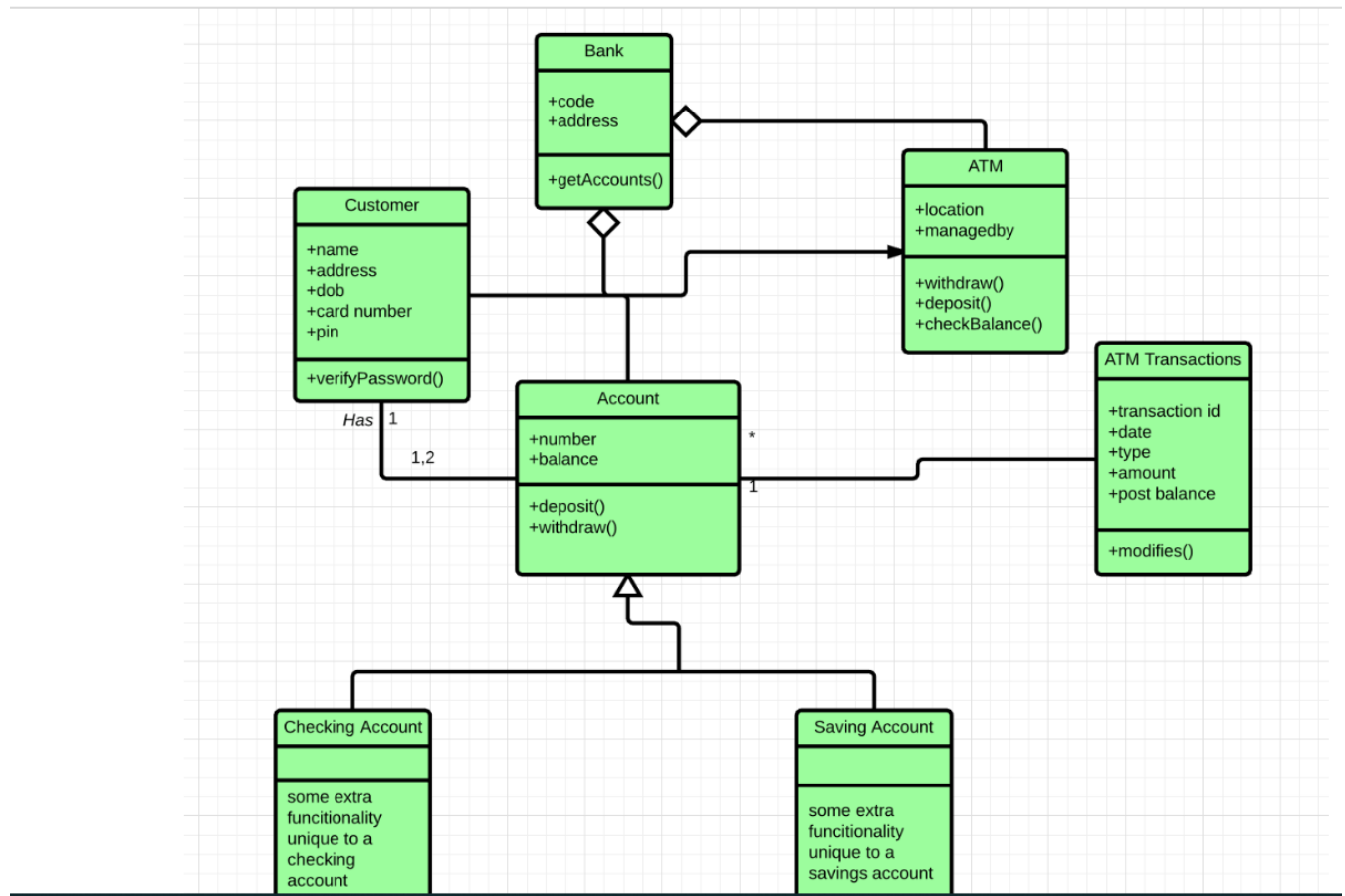
The bank offers 2 types of accounts: a savings account, or a checking account. Both types of accounts will have some functionality unique to them. You can write any dummy functions to represent these unique functionalities.

For any customer, the bank maintains a record of their name, address, date of birth, ATM card number, and PIN. The customer has to verify the password while accessing their account.

As described, banks also maintain ATM booths. Every ATM booth would have a location, and is managed by an official. Customers can withdraw money from the ATM or check their account balance. Every time money is withdrawn, a transaction is said to have taken place. An ATM transaction has a transaction ID, date type (withdraw/check balance), amount, and balance post the transaction.

Based on this description, draw a UML class diagram for the banking system.

SOLUTION:



Some clarifications regarding the UML class diagram:

- 1) The system can be modeled in slightly different ways based on interpretations, and this is in no way the only correct solution. However, it describes the classes required, and one possible set of relationships between these classes.
- 2) Some redundant attributes might have been shown in the diagram. However, the main challenge is to identify the classes and their relationships. Attributes can vary based on requirements.
- 3) These classes show attributes as public (+). Ideally, you should try to make them private (-) wherever possible.
- 4) This diagram shows multiplicities only for bidirectional association relationships. For other classes, you basically need to keep in mind that one bank will be related to multiple accounts, ATM booths and customers.
- 5) This diagram is for a rudimentary banking system. If this system becomes more complex, you can add more classes and model their relationships accordingly.

Note: As an extra exercise, you can attempt to draw the use case diagram for this banking system.